

MANUAL DE REFERENCIA DE CÓDIGO: SISTEMA DE BIBLIOTECA Y PRÉSTAMOS UDI

1. MÓDULOS DE MODELOS DE DATOS (Persistencia JSON)

1.1 Archivo: `Libro.py`

Este archivo define la entidad básica de un libro y la clase encargada de administrar el ciclo de vida de los datos (CRUD) y su almacenamiento persistente en formato JSON.

Clase `Libro`

- **¿Qué hace?:** Define el constructor de la entidad con las propiedades esenciales de un texto académico.
- **¿Para qué sirve?:** Sirve para instanciar objetos tipo `Libro` dentro de la memoria dinámica de la aplicación antes de ser guardados o procesados.

```
class Libro:
    """Clase que representa la entidad de un Libro."""
    def __init__(self, id_libro, nombre, editorial, autor, copias, fecha):
        self.id = id_libro
        self.nombre = nombre
        self.editorial = editorial
        self.autor = autor
        self.copias = copias
        self.fecha = fecha
```

Clase `GestionLibros` (Métodos de Carga y Guardado)

- **¿Qué hace?:** Inicializa la ruta del archivo de almacenamiento (`data/libros.json`) y provee las funciones privadas `_cargar_desde_json` y `_guardar_en_json`.
- **¿Para qué sirve?:** Permite que el sistema sea persistente. `_cargar_desde_json` deserializa los diccionarios del JSON convirtiéndolos de nuevo en objetos `Libro`, mientras que `_guardar_en_json` serializa el estado actual usando el atributo mapeado `__dict__`.

```
class GestionLibros:
    """Clase encargada de la lógica de negocio y almacenamiento en archivo JSON."""
    def __init__(self, archivo_json="data/libros.json"):
        self.archivo_json = archivo_json
        self.lista_libros = []
        # Cargar los datos guardados inmediatamente al iniciar la app
        self._cargar_desde_json()

    def _cargar_desde_json(self):
        """Lee el archivo JSON y reconstruye el array dinámico con objetos Libro."""
        if os.path.exists(self.archivo_json):
            try:
                with open(self.archivo_json, "r", encoding="utf-8") as f:
                    datos = json.load(f)
                    self.lista_libros = [
                        Libro(item['id'], item['nombre'], item['editorial'], item['autor'],
                             for item in datos
                        ]
            except Exception:
                self.lista_libros = []

    def _guardar_en_json(self):
        """Convierte el array dinámico a diccionarios y lo vuelca en el archivo JSON."""
        try:
            with open(self.archivo_json, "w", encoding="utf-8") as f:
                # Convertimos la lista de objetos a una lista de diccionarios con __dict__
                datos_dict = [libro.__dict__ for libro in self.lista_libros]
                json.dump(datos_dict, f, ensure_ascii=False, indent=4)
        except Exception as e:
            print(f"Error al escribir en el JSON: {e}")
```

Métodos de Lógica de Negocio (**guardar**, **editar**, **eliminar**, **actualizar_copias**)

- **¿Qué hace?:** Controla las reglas de negocio en la memoria interna (listas de Python) y gatilla el guardado en disco:
 - **guardar**: Válida la no duplicidad de IDs antes de registrar.
 - **editar** y **eliminar**: Buscan coincidencias por ID para alterar o remover registros.
 - **actualizar_copias**: Resta o suma existencias dinámicamente cuando ocurre un préstamo o devolución.
- **¿Para qué sirve?** Mantiene la integridad de los datos de inventario sin mezclar lógica de persistencia directamente en las pantallas de la interfaz gráfica.

```
def guardar(self, nuevo_libro):
    """Agrega un nuevo libro al array y actualiza el JSON validando que el ID no se rep
    for libro in self.lista_libros:
        if libro.id == nuevo_libro.id:
            return False, f"Error: El ID '{nuevo_libro.id}' ya existe."

    self.lista_libros.append(nuevo_libro)
    self._guardar_en_json() # Guardado persistente
    return True, "Libro guardado exitosamente."

def editar(self, id_libro, nombre, editorial, autor, copias, fecha):
    """Busca un libro por su ID original, actualiza sus campos y lo guarda en el JSON."
    for libro in self.lista_libros:
        if libro.id == id_libro:
            libro.nombre = nombre
            libro.editorial = editorial
            libro.autor = autor
            libro.copias = copias
            libro.fecha = fecha
            self._guardar_en_json() # Guardado persistente
            return True, "Libro modificado exitosamente."

    return False, "Error: Libro no encontrado para editar."

def eliminar(self, id_libro):
    """Elimina un libro del array dinámico y del archivo JSON buscando por su ID."""
    for libro in self.lista_libros:
        if libro.id == id_libro:
            self.lista_libros.remove(libro)
            self._guardar_en_json() # Guardado persistente
            return True, "Libro eliminado exitosamente."

    return False, "Error: Libro no encontrado para eliminar."

def actualizar_copias(self, id_libro, delta):
    """Suma delta (positivo o negativo) a las copias del libro con el id dado."""
    for libro in self.lista_libros:
        if libro.id == id_libro:
            libro.copias = str(max(0, int(libro.copias) + delta))
            self._guardar_en_json()
            return True

    return False
```

1.2 Archivo: Prestamo.py

Contiene la estructura de datos para el control transaccional de los libros prestados a los alumnos.

Clase Prestamo y Lógica de Multas/Devoluciones

- ¿Qué hace?: Mapea campos clave como fechas de entrega y estados ("Activo"/"Inactivo"). En su gestor (`GestionPrestamos`), calcula de manera dinámica los días de retraso usando `datetime.strptime` comparando la fecha de vencimiento contra la fecha actual (`datetime.now()`).
- ¿Para qué sirve?: Permite auditar qué alumnos tienen libros retenidos e inactivar los préstamos una vez devueltos a las estanterías de la universidad.

```
class Prestamo:
    def __init__(self, codigo_estudiante, nombre_estudiante, id_libro, nombre_libro,
                  cantidad, fecha_prestamo, fecha_devolucion, dias_multa, estado):
        self.codigo_estudiante = codigo_estudiante
        self.nombre_estudiante = nombre_estudiante
        self.id_libro = id_libro
        self.nombre_libro = nombre_libro
        self.cantidad = cantidad
        self.fecha_prestamo = fecha_prestamo
        self.fecha_devolucion = fecha_devolucion
        self.dias_multa = dias_multa
        self.estado = estado

class GestionPrestamos:
    def __init__(self, archivo_json="data/prestamos.json"):
        self.archivo_json = archivo_json
        self.lista_prestamos = []
        self._cargar_desde_json()

    def _cargar_desde_json(self):
        if os.path.exists(self.archivo_json):
            try:
                with open(self.archivo_json, "r", encoding="utf-8") as f:
                    datos = json.load(f)
                    self.lista_prestamos = [
                        Prestamo(
                            item['codigo_estudiante'], item['nombre_estudiante'],
                            item['id_libro'], item['nombre_libro'], item['cantidad'],
                            item['fecha_prestamo'], item['fecha_devolucion'],
                            item['dias_multa'], item['estado']
                        )
                        for item in datos
                    ]
            except Exception:
                self.lista_prestamos = []

    def _guardar_en_json(self):
        try:
            with open(self.archivo_json, "w", encoding="utf-8") as f:
                json.dump([p.__dict__ for p in self.lista_prestamos], f,
                          ensure_ascii=False, indent=4)
        except Exception as e:
            print(f"Error al escribir en el JSON: {e}")

    def registrar(self, nuevo_prestamo):
        self.lista_prestamos.append(nuevo_prestamo)
        self._guardar_en_json()
        return True, "Préstamo registrado exitosamente."
```

```
def devolver(self, codigo_estudiante, id_libro):
    for prestamo in self.lista_prestamos:
        if (prestamo.codigo_estudiante == codigo_estudiante and
            prestamo.id_libro == id_libro and
            prestamo.estado == "Activo"):
            try:
                fecha_d = datetime.strptime(prestamo.fecha_devolucion, "%d/%m/%Y")
                hoy = datetime.now()
                prestamo.dias_multa = str(max(0, (hoy.date() - fecha_d.date()).days))
                if hoy.date() > fecha_d.date() else "0"
            except Exception:
                prestamo.dias_multa = "0"
            prestamo.estado = "Inactivo"
            self._guardar_en_json()
            return True, "Devolución registrada exitosamente."
    return False, "No se encontró un préstamo activo con esos datos."

def eliminar(self, codigo_estudiante, id_libro):
    """Elimina el préstamo activo del array y del JSON."""
    for prestamo in self.lista_prestamos:
        if (prestamo.codigo_estudiante == codigo_estudiante and
            prestamo.id_libro == id_libro and
            prestamo.estado == "Activo"):
            self.lista_prestamos.remove(prestamo)
            self._guardar_en_json()
            return True, "Préstamo eliminado correctamente."
    return False, "No se encontró un préstamo activo con esos datos."

def obtener_todos(self):
    self._cargar_desde_json()
    return self.lista_prestamos
```

2. COMPONENTES DE LA INTERFAZ GRÁFICA (GUI con Tkinter)

2.1 Pantalla Principal de Inventarios: `biblioteca.py`

Esta clase hereda de `tk.Frame` y construye toda la administración de libros.

SISTEMA DE GESTIÓN DE BIBLIOTECA UDI

FECHA DE INGRESO

ID LIBRO

NOMBRE DEL LIBRO

EDITORIAL

AUTOR

NUMERO DE COPIAS

BUSCAR (ID o Nombre):

Id Libro	Nombre Libro	Editorial	Autor	Num. copias	Fecha Ingreso
----------	--------------	-----------	-------	-------------	---------------

Captura la ventana de la aplicación ejecutándose en la sección de Biblioteca, mostrando las cajas de texto y el Treeview inferior.

Inicialización de Componentes y Validaciones

- **¿Qué hace?:** Modela visualmente las cajas de texto (`tk.Entry`) con límites estrictos de caracteres mediante la función `limitar_caracteres` acoplada al evento `<KeyRelease>`, y fuerza restricciones numéricas en el campo de copias a través de la función nativa `register` con `validar_numeros`.
- **¿Para qué sirve?:** Previene errores en la entrada de datos por parte del usuario (por ejemplo, escribir letras en la cantidad de copias o exceder el tamaño límite de almacenamiento).

```
def validar_numeros(self, texto):  
    if texto == "" or texto.isdigit():  
        return True  
    return False  
  
def limitar_caracteres(self, entrada, max_caracteres):  
    contenido = entrada.get()  
    if len(contenido) > max_caracteres:  
        entrada.delete(max_caracteres, tk.END)
```

Interacción con el Treeview (`Grilla`) y Búsquedas

- **¿Qué hace?:** Lee colecciones completas mediante `mostrar_datos_mock` o segmentadas por criterios lógicos de filtrado en el método `buscar_libro`.

- **¿Para qué sirve?:** Renderiza de manera tabular la base de datos de libros directamente en pantalla para que el operador de la biblioteca visualice el catálogo en tiempo real.

```
def buscar_libro(self):
    termino_búsqueda = self.ent_buscar.get().strip().lower()

    if not termino_búsqueda:
        messagebox.showwarning("Búsqueda vacía", "Por favor, ingrese un ID o Nombre de libro para realizar la búsqueda.")
        return

    for item in self.grilla.get_children():
        self.grilla.delete(item)

    todos_los_libros = self.controller.gestion_libros.obtener_todos()
    encontrado = False

    for libro in todos_los_libros:
        if termino_búsqueda in libro.id.lower() or termino_búsqueda in libro.nombre.lower():
            self.grilla.insert("", "end", values=(
                libro.id,
                libro.nombre,
                libro.editorial,
                libro.autor,
                libro.copias,
                libro.fecha
            ))
            encontrado = True

    if not encontrado:
        messagebox.showinfo("Sin resultados", f"No se encontró ningún libro con el criterio: '{termino_búsqueda}'")
```

2.2 Pantalla Transaccional: [prestamos.py](#)

Controla el flujo para asignar ejemplares a los estudiantes y procesar sus retornos mediante ventanas emergentes.

SISTEMA DE GESTIÓN DE BIBLIOTECA UDI

SISTEMA DE PRÉSTAMOS DE BIBLIOTECA UDI

CÓDIGO ESTUDIANTE NOMBRES Y APELLIDOS

LIBROS DISPONIBLES

Id Libro	Nombre Libro	Num. Copias
1	guayacan	11
3	pppp	1

BUSCAR PRÉSTAMO (Cód. Estudiante):

PRÉSTAMOS REGISTRADOS (clic para ver detalle)

Cód. Estudiante	Nombre	Id Libro	Nombre Libro	Cantidad	F. Préstamo	F. Devolución	Días Multa	Estado
1111	juan	1	guayacan	1	29/05/2026	13/06/2026	0	●
222	ada	3	pppp	1	29/05/2026	13/06/2026	0	●

Captura la sección de préstamos donde se observan las filas de colores (verdes/rojas) y el modal flotante de detalles.

Reglas Lógicas de Alertas Visuales (Tags de Color)

- **¿Qué hace?:** Evalúa las fechas límites mediante `calcular_indicador`. Si un libro superó la fecha de devolución establecida, le inyecta la etiqueta "rojo"; de lo contrario, aplica "verde".
- **¿Para qué sirve?:** Sirve como un sistema de semáforo visual para alertar de inmediato al usuario si un estudiante posee un préstamo vencido.

```
def calcular_indicador(self, estado_modelo, fecha_devolucion_str):
    """Devuelve (texto_estado, tag) según si el préstamo está vencido o no."""
    if estado_modelo == "Inactivo":
        return "●", "verde"
    try:
        fecha_d = datetime.strptime(fecha_devolucion_str, "%d/%m/%Y")
        if datetime.now().date() > fecha_d.date():
            return "●", "rojo"
    except Exception:
        pass
    return "●", "verde"
```

Control del Modal de Devolución (`abrir_modal_prestamo`)

- **¿Qué hace?:** Instancia un contenedor de nivel superior flotante (`tk.Toplevel`) usando `grab_set()` para bloquear la ventana de fondo temporalmente mientras se decide realizar la devolución física del recurso.

- **¿Para qué sirve?:** Proporciona un desglose limpio y detallado de la transacción activa y permite liberar las copias del libro sumándolas de vuelta al stock.

```
def abrir_modal_prestamo(self, event):
    seleccion = self.tabla_prestamos.selection()
    if not seleccion:
        return

    item_id = seleccion[0]
    valores = self.tabla_prestamos.item(item_id, "values")
    estado_modelo = self._estado_modelo.get(item_id, "Inactivo")

    modal = tk.Toplevel(self)
    modal.title("Detalle del Préstamo")
    modal.geometry("380x260")
    modal.resizable(False, False)
    modal.configure(bg="white")
    modal.update_idletasks()
    modal.grab_set()
```

3. ORQUESTADOR PRINCIPAL: `main.py`

Este archivo es el punto de entrada ejecutable de todo el ecosistema de software.

- **¿Qué hace?:** Instancia la raíz `tk.Tk`, inicializa de manera compartida las clases controladoras de datos (`GestionLibros` y `GestionPrestamos`), y dibuja las pantallas usando un diseño superpuesto controlado mediante `tkraise()`.
- **¿Para qué sirve?:** Sirve como el enrutador central de la aplicación (arquitectura basada en controladores), garantizando que las dos pantallas compartan exactamente la misma información en tiempo real sin abrir ventanas independientes repetitivas.

```
class BibliotecaApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("SISTEMA DE GESTIÓN DE BIBLIOTECA UDI")
        self.geometry("900x700")
        self.resizable(False, False)

        self.gestion_libros = GestionLibros()
        self.gestion_prestamos = GestionPrestamos()

        self.contenedor = tk.Frame(self)
        self.contenedor.pack(fill="both", expand=True)
        self.contenedor.grid_rowconfigure(0, weight=1)
        self.contenedor.grid_columnconfigure(0, weight=1)

        self.pantalla_biblioteca = BibliotecaPantalla(parent=self.contenedor, controller=self)
        self.pantalla_biblioteca.grid(row=0, column=0, sticky="nsew")

        self.pantalla_prestamos = PrestamosPantalla(parent=self.contenedor, controller=self)
        self.pantalla_prestamos.grid(row=0, column=0, sticky="nsew")

        self.mostrar_pantalla("biblioteca")
        self.pantalla_biblioteca.actualizar_fecha()

    def mostrar_pantalla(self, nombre):
        if nombre == "biblioteca":
            self.pantalla_biblioteca.tkraise()
        elif nombre == "prestamos":
            self.pantalla_prestamos.tkraise()
            self.pantalla_prestamos.cargar_libros()
            self.pantalla_prestamos.cargar_prestamos()
```